Introducción (nueva)

Actualmente, la gran cantidad de datos generados por diversas fuentes ha convertido el análisis de datos en un campo básico de estudio para obtener información funcional y ayudar a la toma de decisiones fundamentales. En esta área se combinan distintas técnicas de estadística, aprendizaje automático al igual que la visualización para descubrir patrones, conexiones ocultas y tendencias dentro de grandes volúmenes de datos. En este contexto en específico, este proyecto se enfoca en la aplicación práctica de los algoritmos de aprendizaje, de los cuales se dividen en dos tipos: supervisado y no supervisado, con el propósito de analizar una serie de datos sobre frijoles secos. Python y algunas bibliotecas especializadas fueron utilizadas y se implementaron métodos de clasificación como Naive Bayes y redes neuronales artificiales, al igual que técnicas de agrupamiento como K-Means, DBSCAN y agrupamiento jerárquico.Este enfoque no solo permite explorar la estructura latente de los datos sin etiquetas, sino también nos permite comparar el rendimiento de los modelos generados para las clases reales. De esta forma, el propósito del estudio es indicar la utilidad práctica de la minería de datos para resolver problemas reales, lo que a su vez fortalece la comprensión teórica y técnica de los métodos utilizados.

Asimismo, verificar los diversos enfoques y la clasificación de los datos, esta aplicación tiene como objetivo mostrar las relaciones que se pueden determinar entre logrados a través de algoritmos no supervisados y su uso posterior en los modelos monitoreados. Mediante esta estrategia, se contempla cómo, a pesar de la ausencia de etiquetas reales, es posible generar clasificaciones funcionales y confiables que permiten entrenar modelos con alta precisión. Como último recurso, el objetivo es mostrar el valor que tienen estos métodos al instante de la toma de decisiones con base en los datos, tanto en el entorno académico como en los escenarios de aplicación real.

Desde un punto de vista académico, este proyecto refleja el puente entre el conocimiento teórico adquirido en la clase y su uso en escenarios de la vida cotidiana. La utilización de datos comprobados nos da una dimensión práctica que forza a enfrentar desafíos como la limpieza de los datos, el procesamiento de valores atípicos, la selección de mediciones apropiadas e interpretación crítica de los resultados. Todos estos elementos mencionados mejoran el aprendizaje y la preparación de nosotros como estudiantes para los desafíos que enfrentamos en las ciencias de los datos modernas. Aunque la clasificación de Bean puede parecer una tarea fácil, la estructura multidimensional y las diferencias en las clases lo convierten en un caso de estudio informativo para la evaluación del rendimiento de varios algoritmos tanto supervisados como no supervisados.

Aparte del uso de herramientas como Scikit Learning, Pandas, NumPy y Seaborn en el entorno de Python brinda un marco de trabajo poderoso y versátil. Estas bibliotecas le permiten manipular datos, usar algoritmos y visualizar efectivamente los resultados. De esta forma, nosotros como estudiantes no solo comprendemos la teoría, sino que también nos enfrentamos a desafíos reales como el manejo de grandes volúmenes de información

Uno de los aspectos más valiosos del estudio es la capacidad de comparar el rendimiento entre las etiquetas reales y las generadas mediante técnicas de agrupamiento. Esta comparación no solo proporciona información cuantitativa utilizando mediciones como la

precisión y los índices F1, sino que también nos permite reflexionar sobre la capacidad de los métodos no supervisados para acercarse a la verdad.

Así, se demuestra que la minería de datos, cuando se aplica con criterio y rigurosidad, se convierte en una herramienta esencial para comprender fenómenos, optimizar procesos y apoyar decisiones en una amplia variedad de campos.

Por tanto, se demuestra que la minería de datos, cuando se utiliza con criterio y de manera rigurosa, se convierte en una herramienta indispensable y útil para comprender los fenómenos, para reducir los procesos y las decisiones en on on una amplia gama de campos.

1. Minería de datos

"La minería de datos es el uso del machine learning y el análisis estadístico para descubrir patrones y otra información valiosa de grandes conjuntos de datos" (IBM, s. f., párr. 1).

Actualmente se vive un constante avance sobre el aprendizaje automático, el almacenamiento masivo de datos y el crecimiento del big data han acelerado la adaptación de la minería de datos, conocida como descubrimiento de conocimiento de en bases de datos (KDD).

Esta metodología se vuelve elemental para revelar patrones no evidentes, hacer predicciones y apoyar a la toma de decisiones en diferentes organizaciones. Su empleo se ha difundido en diversas áreas como lo es el sector salud, finanzas, publicidad, entre muchos.

La **Minería de Datos** es una técnica que nos ayuda a facilitar la organización, filtrado y el análisis de datos sofisticados con el fin de detectar tendencias significativas, comportamientos de los usuarios, estafas, cuellos de botella o posibles amenazas a la seguridad. Todo esto mediante la ayuda de algoritmos de machine learning e inteligencia artificial, lo que nos permite que el análisis pueda ser automatizado y optimizado. Tal como lo señala Holdsworth (2024): “La minería de datos toma los datos en bruto y encuentra el orden en el caos: ver el bosque por los árboles”.

Entre las principales ventajas de la Minería de Datos se encuentran la habilidad de identificar conceptos ocultos, mejorar procesos empresariales y abordar diversos retos analíticos en diferentes áreas de la organización.Sin embargo, también nos encontramos con importantes obstáculos, tales como la complejidad técnica, la necesidad de contar con personal especializado, los altos costos asociados a la recolección y almacenamiento de datos, así como la posibilidad de que los resultados sean ambiguos o incluso erróneos.

Existen múltiples técnicas que se pueden utilizar para realizar Minería de Datos. Entre ellas destacan la clasificación, el agrupamiento, las reglas de asociación, los árboles de decisión y el método de vecinos más cercanos (KDD). Cada técnica permite extraer información de distintas maneras, ya sea mediante la organización de elementos similares, la creación de vínculos entre variables o la predicción de resultados a partir de datos anteriores.

En conclusión, la minería de datos se establece como una área fundamental para transformar datos en información valiosa. Sin embargo, es esencial llevar a cabo una gestión ética y responsable, así como realizar un análisis crítico de los resultados alcanzados, teniendo en cuenta que la correlación no necesariamente indica causalidad.

1. Aprendizaje No Supervisado

El aprendizaje no supervisado es una rama del machine learning que ayuda a examinar datos no etiquetados para descubrir patrones ocultos o agrupaciones naturales en la información. En comparación con el aprendizaje supervisado este método no necesita la participación de personas para categorizar o etiquetar los datos, lo que lo hace una herramienta perfecta para el análisis exploratorio, segmentación de clientes, identificación de imágenes y descubrimiento de irregularidades.

Los métodos más habituales incluyen la agrupación de clústeres, las reglas de asociación y la reducción de dimensionalidad. En el proceso de agrupación, la información se estructura en grupos o clústeres basándose en sus semejanzas, empleando algoritmos como k-means, modelos jerárquicos y probabilísticos como los modelos de mezcla gaussiana (GMM).

Las reglas de asociación, como el algoritmo Apriori, se utilizan para detectar vínculos comunes entre elementos, resultando práctico para motores de búsqueda y análisis de comportamientos de consumo. En cambio las técnicas de como PCA, SVD o codificadores automáticos se emplean para disminuir el número de variables en un conjunto de datos, optimizando el desempeño del modelo y simplificando su visualización.

Este tipo de aprendizaje, por su habilidad para identificar patrones sin supervisión, se utiliza en varias áreas como la medicina, la publicidad, el e-commerce o los sistemas de recomendación. Sin duda, el aprendizaje no supervisado es un recurso esencial en la época del big data, ya que posibilita obtener conocimiento valioso de la información en bruto.

4. Objetivo del trabajo:

El objetivo principal de este trabajo colaborativo es poner en práctica los conocimientos obtenidos sobre Minería de Datos, particularmente en los contextos del aprendizaje supervisado y no supervisado para el estudio de un conjunto de datos reales vinculados a los tipos de frijoles secos que existen. Con este dataset, se busca aplicar y analizar diversos algoritmos de clasificación y agrupación con el objetivo de reconocer, distinguir y clasificar cada clase de frijoles de manera exacta y eficaz.

El proyecto tiene como objetivo ampliar la comprensión de los algoritmos tradicionales de clasificación supervisada, además de técnicas de agrupación no supervisada. Para lograrlo, se emplea el lenguaje de programación Python, que proporciona un extenso ecosistema

de bibliotecas enfocadas en la minería de datos y al machine learning, lo que posibilita experimentar con diversas técnicas de manera programada, adaptable y repetible.

En resumen, la finalidad de este estudio no solo consiste en implementar los principios básicos de aprendizaje supervisado y no supervisado, junto con los índices de validación de agrupamiento, sino también en evidenciar su utilidad práctica al examinar un conjunto de datos reales acerca de los tipos de frijoles secos. Mediante la utilización de Python como herramienta principal, se pretende demostrar cómo estos métodos facilitan la identificación de patrones encubiertos, la valoración de la calidad de los modelos producidos y el fortalecimiento de la toma de decisiones basada en datos, consolidando de esta manera los conocimientos obtenidos durante el curso.

Marco Teórico

Algoritmos de aprendizaje no supervisado

Dentro del ámbito del aprendizaje automático, los algoritmos de aprendizaje no supervisado representan un conjunto de técnicas que facultan a las máquinas para discernir patrones y estructuras intrínsecas en volúmenes de datos que carecen de etiquetado previo. En contraposición al aprendizaje supervisado, el cual se fundamenta en la provisión de ejemplos con sus correspondientes soluciones, el aprendizaje no supervisado opera sobre información no clasificada, posibilitando que el modelo identifique autónomamente las relaciones y agrupaciones subyacentes en los datos.

De acuerdo con Naeem et al. (2023), la metodología del aprendizaje no supervisado suprime la exigencia de contar con datos previamente etiquetados y la intervención manual en la ingeniería de características. Esta característica confiere a los métodos de aprendizaje automático una mayor versatilidad y un incremento en su nivel de automatización.

El algoritmo K-means

El algoritmo K-means constituye un método de partición cuyo objetivo primordial reside en la asignación de cada observación perteneciente a un conjunto de datos a uno de los K clústeres predefinidos. Dicha asignación se realiza de tal modo que se minimice la suma de las distancias euclidianas elevadas al cuadrado entre cada punto y el centroide del clúster al que ha sido asociado.

Procedimiento Detallado y Descriptivo del Algoritmo K-means

1. Inicialización de los centroides: La etapa inicial del algoritmo implica la selección de K puntos que actuarán como los centroides primarios de cada uno de los clústeres. Esta selección puede llevarse a cabo mediante diversas estrategias, siendo dos de las más comunes:
    - Selección aleatoria: Consiste en la elección arbitraria de K puntos del conjunto de datos para servir como centroides iniciales.
    - Método K-means++: Opta por una selección de los centroides iniciales de manera más estratégica, con el propósito de prevenir una concentración excesiva de los puntos iniciales en una región limitada del espacio de datos. Esta aproximación contribuye a mejorar la estabilidad y la calidad del resultado final.

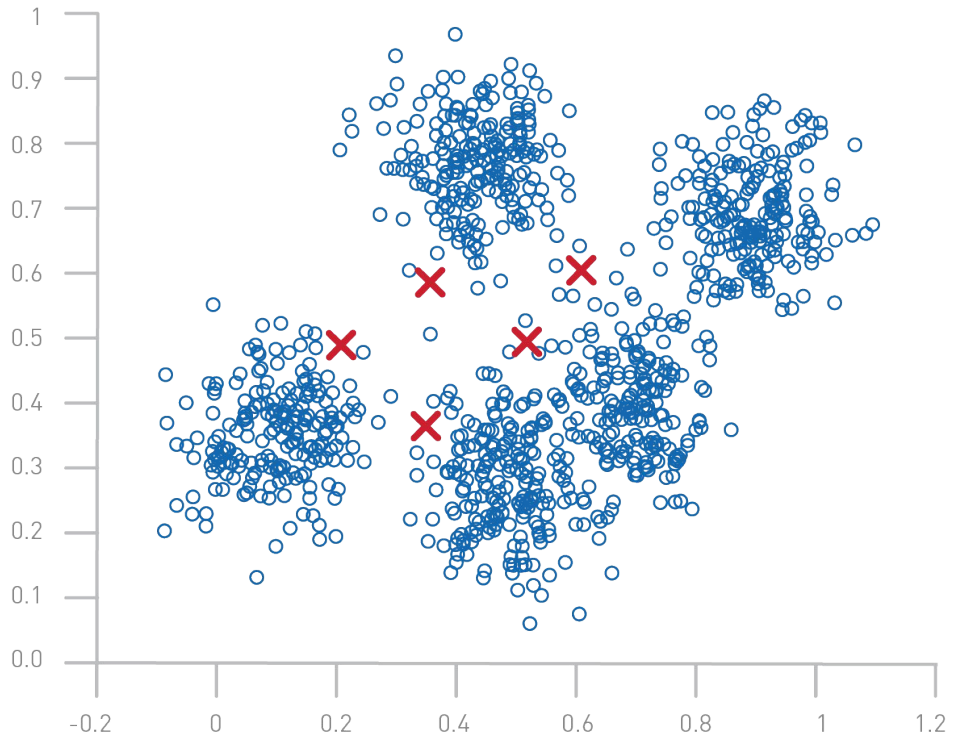
2. Asignación de puntos a clústeres (etiquetado): Esta fase responde a la cuestión de la pertenencia de cada punto del conjunto de datos a un clúster específico. El procedimiento se desarrolla de la siguiente manera:

- Para cada punto xi en el conjunto de datos:
- Se calcula la distancia entre dicho punto y cada uno de los K centroides. La métrica de distancia predominante es la distancia euclidiana, que en un espacio bidimensional se expresa como:

$$d(xi,\mu k) = \sqrt{(xi(1)-\mu k(1))2 + (xi(2)-\mu k(2))2}$$

donde μk representa el centroide del clúster k

- Se comparan todas las distancias calculadas y se asigna el punto xi al clúster cuyo centroide presenta la menor distancia.



NVIDIA. (s.f.). *K-Means*.

3. Actualización de los centroides: Una vez que todos los puntos han sido asignados a un clúster, se procede al recálculo de la posición de cada centroide.. El método para llevar a cabo esta actualización es el siguiente:

- Para cada clúster Ck, se calcula la media aritmética de todos los puntos que pertenecen a dicho clúster, según la siguiente fórmula:

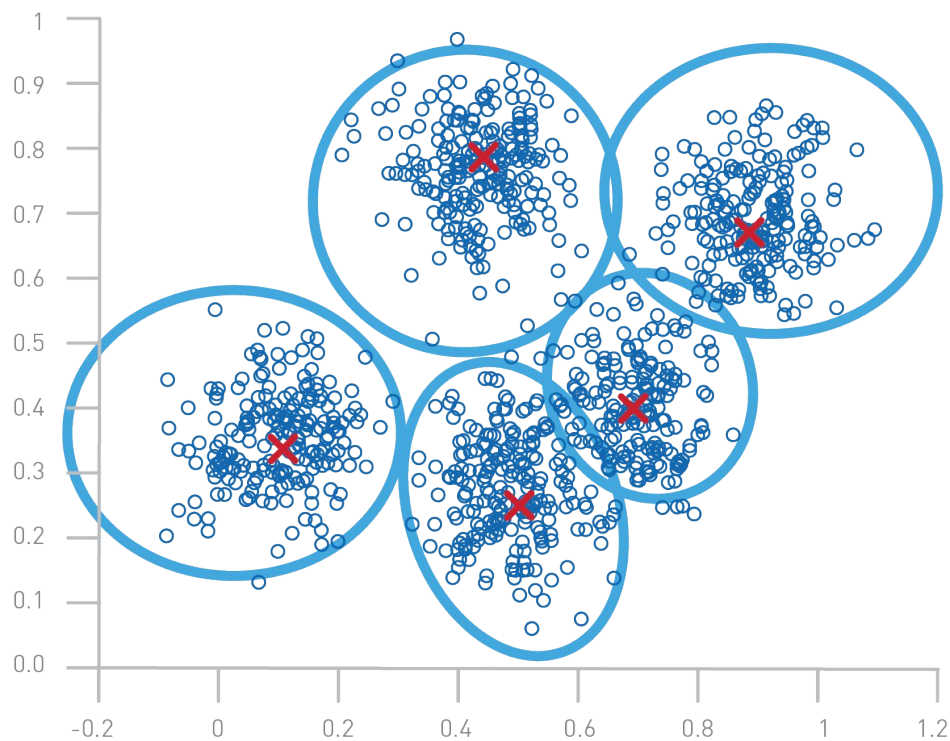
$$\mu k = \frac{1}{|Ck|} \sum_{xi \in Ck} xi$$

donde |Ck| denota el número de puntos contenidos en el clúster k.

Repetición hasta convergencia: El proceso iterativo de asignación de puntos y recalculación de centroides se repite de forma sucesiva. En cada iteración:

- Los puntos son reasignados al centroide más cercano, cuya ubicación ha podido variar en la iteración precedente.
- Los centroides son nuevamente recalculados basándose en la nueva composición de los clústeres.
- Se evalúa si los centroides han experimentado un cambio significativo o si han permanecido estables. Alternativamente, se puede establecer un número máximo de iteraciones como criterio de detención.

NVIDIA. (s.f.). *K-Means*.



Criterios de detención del algoritmo:

- Cuando las asignaciones de los puntos a los clústeres no sufren modificaciones.
- Cuando las nuevas ubicaciones de los centroides son idénticas o presentan una variación mínima respecto a las anteriores.

- Al alcanzar un número predefinido de iteraciones.

El Agrupamiento Jerárquico Aglomerativo

El Análisis Jerárquico Aglomerativo (AHC) es una técnica de clustering que organiza los datos en una estructura jerárquica, representada visualmente mediante un dendrograma. Este método opera mediante la fusión iterativa de los objetos más similares, comenzando con cada objeto como un clúster individual. **1. Funcionamiento del Método**

Conceptos Clave:

- **Distancia Euclidiana:** Una métrica fundamental para cuantificar la similitud (o disimilitud) entre dos objetos en un espacio multidimensional. Se calcula como la raíz cuadrada de la suma de las diferencias al cuadrado entre sus correspondientes atributos:

  $$d(Oi,Oj) = \sqrt{(xi-xj)^2 + (yi-yj)^2 + \ldots}$$

  donde Oi y Oj son dos objetos, y x,y,… representan sus atributos.

- **Matriz de Distancias:** Una tabla simétrica que almacena las distancias calculadas entre todos los pares posibles de objetos en el conjunto de datos. Esta matriz sirve como base para las decisiones de fusión durante el proceso de agrupamiento.

2. Procedimiento

1. Preparación de Datos

- **Objetos:** Se tiene un conjunto de n objetos, denotados como O1,O2,…,On.
- **Atributos:** Cada objeto está descrito por m atributos. Por ejemplo, el objeto O1 podría representarse como un vector de atributos (x1,y1,z1).

2. Calcular Matriz de Distancias

- **Métrica:** Se utiliza una métrica de distancia para cuantificar la similitud (o disimilitud) entre cada par de objetos. La distancia euclidiana es una opción común para atributos continuos:

  $$d(Oi,Oj) = \sqrt{\sum_{k=1}^{m} (Oik-Ojk)2}$$

  donde Oik representa el valor del k-ésimo atributo del objeto Oi. Se pueden utilizar otras métricas como la distancia de Manhattan o la distancia de Minkowski, dependiendo de la naturaleza de los datos.

- **Matriz simétrica:** Se construye una matriz de distancias de dimensiones n×n, donde el elemento Dij representa la distancia entre el objeto Oi y el objeto Oj, es decir, Dij=d(Oi,Oj). Esta matriz es simétrica (Dij=Dji) y tiene ceros en la diagonal (Dii=0).

3. Inicialización

- Inicialmente, cada objeto se considera como un clúster individual. Por lo tanto, se tienen n clústeres: C1={O1},C2={O2},…,Cn={On}.

4. Proceso Iterativo de Fusión

Este proceso se repite hasta que se cumple un criterio de terminación (ver paso 6). En cada iteración:

- **Identificar clústeres más cercanos:** Se examina la matriz de distancias actual para encontrar el par de clústeres Ca y Cb cuya distancia d(Ca,Cb) es mínima. La forma en que se calcula la distancia entre dos clústeres depende del criterio de enlace seleccionado (ver paso 5).
- **Fusionar clústeres:** Los clústeres Ca y Cb identificados como los más cercanos se fusionan para formar un nuevo clúster Cab=Ca∪Cb.
- **Actualizar matriz de distancias:**
- Se eliminan las filas y columnas correspondientes a los clústeres Ca y Cb de la matriz de distancias.
- Se añade una nueva fila y columna a la matriz para representar el nuevo clúster Cab. Las distancias entre el nuevo clúster Cab y los demás clústeres restantes se recalculan según el criterio de enlace elegido.

5. Criterios de Enlace (Ejemplos)

Los criterios de enlace definen cómo se mide la distancia entre dos clústeres que contienen uno o más objetos:

- **Single Linkage (Enlace Simple):** La distancia entre dos clústeres Cx y Cy es la mínima distancia entre cualquier par de objetos, donde un objeto pertenece a Cx y el otro a Cy: \

  $$d(Cx,Cy) = min\{d(Oi,Oj) \mid Oi \in Cx, Oj \in Cy\}$$

- **Complete Linkage (Enlace Completo):** La distancia entre dos clústeres Cx y Cy es la máxima distancia entre cualquier par de objetos, donde un objeto pertenece a Cx y el otro a Cy:

  $$d(Cx,Cy) = max\{d(Oi,Oj) \mid Oi \in Cx, Oj \in Cy\}$$

- **Average Linkage (Enlace Promedio):** La distancia entre dos clústeres Cx y Cy es el promedio de las distancias entre todos los pares de objetos, donde un objeto pertenece a \

  $Cx\ y\ el\ otro\ a\ Cy$: $d(Cx,Cy) =| Cx || Cy | 1Oi{\in}Cx{\sum}Oj{\in}Cy{\sum}d(Oi,Oj)$

6. Terminación

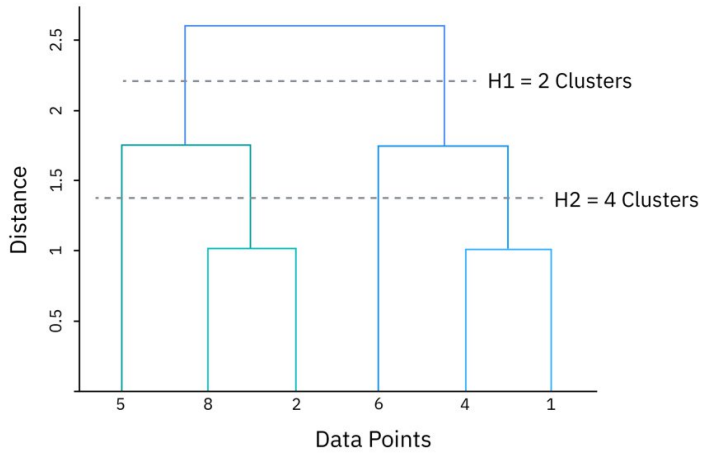
El proceso iterativo de fusión continúa hasta que se cumple una de las siguientes condiciones:

- **Un único clúster:** Todos los objetos han sido agrupados en un único clúster que contiene la totalidad del conjunto de datos.
- **Número predeterminado de clústeres:** Se alcanza un número específico de clústeres deseado. Esto se puede lograr deteniendo el algoritmo cuando quedan k clústeres o cortando el dendrograma a un nivel que genere k ramas principales.

7. Construcción del Dendrograma

El dendrograma es un diagrama de árbol que representa la historia de las fusiones realizadas durante el AHC.

- **Eje X:** En la base del dendrograma se encuentran los objetos individuales. A medida que se asciende en el árbol, las ramas se unen a diferentes alturas, representando las fusiones de clústeres.
- **Eje Y:** La altura a la que dos ramas se unen en el dendrograma indica la distancia a la que se fusionaron los correspondientes clústeres. Las fusiones que ocurren a distancias menores son representadas por uniones en la parte inferior del dendrograma, mientras que las fusiones a distancias mayores se encuentran en la parte superior.
- **Corte:** Para obtener un número específico de clústeres a partir del dendrograma, se puede trazar una línea horizontal a una altura determinada. El número de ramas verticales que interseca esta línea representa el número de clústeres resultantes para esa distancia de fusión. La elección de la altura del corte depende del análisis y del número de clústeres que se considere apropiado para el problema en cuestión.


H1 = 2 Clusters
H2 = 4 Clusters
Distance
0.5
1
1.5
2
2.5
5
8
2
6
4
1
Data Points

El algoritmo DBSCAN

El algoritmo DBSCAN (Density-Based Spatial Clustering of Applications with Noise) es un método de agrupamiento que identifica regiones de alta densidad de puntos en el espacio de datos, agrupándolos en clústeres, y marca como ruido aquellos puntos que residen en regiones de baja densidad.

Procedimiento Detallado del Algoritmo DBSCAN

1. **Definición de parámetros:** El funcionamiento de DBSCAN se fundamenta en la definición de dos parámetros esenciales:

- **ϵ (epsilon):** Define el radio de la vecindad alrededor de un punto dado.
- **MinPts:** Especifica el número mínimo de puntos requeridos dentro de la vecindad ϵ de un punto para que este sea considerado un punto núcleo y pueda iniciar la formación de un clúster.

2. Clasificación de puntos: En base a los parámetros definidos, cada punto en el conjunto de datos es clasificado en una de las siguientes categorías:

- **Punto núcleo:** Un punto es considerado núcleo si contiene al menos MinPts dentro de su vecindad ϵ (incluyéndose a sí mismo).
- **Punto frontera:** Un punto que no es un punto núcleo, pero se encuentra dentro de la vecindad ϵ de al menos un punto núcleo.
- **Punto ruido:** Cualquier punto que no sea ni un punto núcleo ni un punto frontera.

3. Algoritmo paso a paso: El proceso de agrupamiento de DBSCAN se desarrolla de la siguiente manera para cada punto que aún no ha sido visitado:

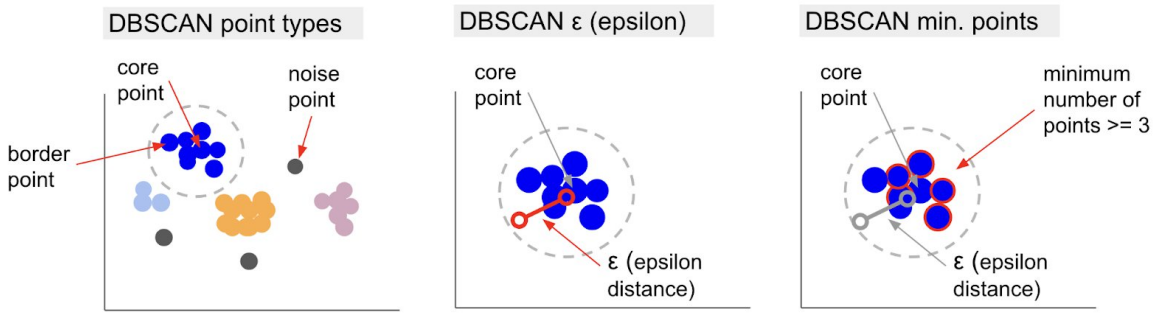
- Marcar el punto como visitado.
- Identificar todos los puntos dentro de su vecindad ϵ.
- Si el número de puntos en su vecindad es al menos MinPts, se inicia la formación de un nuevo clúster y se procede a su expansión:
- Agregar el punto al nuevo clúster.
- Para cada punto dentro de su vecindad:
- Si el punto vecino no ha sido visitado, marcarlo como visitado y obtener su propia vecindad ϵ. Si la vecindad del punto vecino contiene al menos MinPts, agregar todos los puntos de esa vecindad a la vecindad actual del clúster en expansión.
- Si el punto vecino aún no pertenece a ningún clúster, agregarlo al clúster actual.
- Si el número de puntos en la vecindad del punto inicial es menor que MinPts, marcarlo como ruido.

Puntos clave

- **Determinación de parámetros:** La selección apropiada de los valores para ϵ y MinPts es crítica para el rendimiento del algoritmo. Una elección inadecuada puede llevar a resultados de agrupamiento subóptimos. A menudo, la

determinación de estos parámetros requiere experimentación empírica o el uso de métodos como el gráfico del codo (para estimar ϵ en función de las distancias k-ésimas vecinas).

- **Ventajas:** DBSCAN destaca por su capacidad para identificar clústeres con formas complejas o irregulares y por su robustez frente al ruido en los datos.
- **Limitaciones:** El algoritmo es sensible a la elección de los parámetros ϵ y MinPts. Además, puede tener dificultades para detectar clústeres que presentan variaciones significativas en su densidad.



Sampaio (2023), Stack Abuse

Índices de Validez

Los Índices de Validez de Clúster (CVIs) constituyen métricas fundamentales para la evaluación de la calidad inherente a las agrupaciones producidas por algoritmos de clustering. Estos índices posibilitan la determinación tanto de la cohesión interna de los clústeres es decir, el grado de similitud entre los elementos que componen un mismo grupo

Aplicaciones

Los CVIs revisten una importancia capital no solo para la valoración de la calidad de una agrupación dada, sino también para la determinación del número óptimo de clústeres presentes en un conjunto de datos específico. En el contexto de algoritmos de clustering automático que se fundamentan en metaheurísticas, los CVIs son empleados como funciones de aptitud, dirigiendo así el proceso de optimización.

El índice de Dunn

Este constituye una métrica de validación interna ampliamente utilizada en el campo de la minería de datos con el propósito de evaluar la calidad de las particiones generadas por diversos algoritmos de agrupamiento. Propuesto por Joseph C. Dunn en 1974, su objetivo primordial radica en la identificación de aquellas particiones que exhiben clústeres simultáneamente bien separados entre sí y con una alta cohesión interna.

¿Qué es el índice de Dunn?

El índice de Dunn se fundamenta en la maximización de la separación existente entre los distintos clústeres identificados y la minimización de la dispersión de los puntos dentro de cada clúster individual. Formalmente, se define como la razón entre la mínima distancia inter-clúster y el máximo diámetro intra-clúster.

Cálculo del índice de Dunn

Para un conjunto de clústeres C={C1,C2,...,Ck}, el índice de Dunn se calcula mediante la siguiente fórmula:

$$D = \frac{1 \leq i < j \leq k \; mind(Ci,Cj)}{1 \leq l \leq k max\delta(Cl)}$$

Donde:

- d(Ci,Cj): Representa la distancia entre los clústeres Ci y Cj. Comúnmente, esta distancia se define como la mínima distancia entre cualquier par de puntos, donde cada punto pertenece a un clúster diferente.
- δ(Cl): Denota el diámetro del clúster Cl, el cual se define como la máxima distancia existente entre cualquier par de puntos dentro del mismo clúster.

Un valor más elevado del índice D se interpreta como un indicador de una mejor calidad de agrupamiento, caracterizada por clústeres más compactos internamente y mejor segregados entre sí.

Interpretación y uso

- **Valores altos de D:** Sugieren la presencia de clústeres bien definidos, con una elevada cohesión interna (puntos similares dentro del mismo clúster) y una adecuada separación entre ellos (clústeres distintos).
- **Valores bajos de D:** Pueden indicar la existencia de solapamientos entre clústeres o la presencia de clústeres con una dispersión interna considerable.

El índice de Dunn resulta particularmente útil en la determinación del número óptimo de clústeres en un conjunto de datos dado. La estrategia consiste en evaluar el índice para diferentes números de clústeres y seleccionar aquel que arroje el valor máximo de D.

Limitaciones

- **Sensibilidad a valores atípicos:** La presencia de valores atípicos (outliers) en el conjunto de datos puede influir de manera significativa en el cálculo del índice. Estos puntos pueden distorsionar tanto las distancias intra-clúster como las inter-clúster, afectando la interpretación del índice.
- **Complejidad computacional:** El cálculo del índice de Dunn implica la computación de todas las distancias posibles entre pares de clústeres y dentro de cada clúster. Para conjuntos de datos de gran tamaño, esta operación puede resultar computacionalmente costosa en términos de tiempo de procesamiento.
- **Dependencia de la métrica de distancia:** Los resultados del índice de Dunn están intrínsecamente ligados a la elección de la métrica de distancia empleada (por ejemplo, distancia Euclidiana, distancia de Manhattan). Diferentes métricas pueden arrojar valores distintos del índice para la misma partición de datos, lo que subraya la importancia de seleccionar una métrica apropiada para la naturaleza de los datos y los objetivos del análisis.

índice de Davies-Bouldin

El índice de Davies-Bouldin es una métrica de validación interna diseñada para cuantificar la calidad de una estructura de clústeres obtenida mediante un algoritmo de agrupamiento. Su propósito fundamental es evaluar simultáneamente tanto la separación existente entre los diferentes clústeres como la compacidad de los puntos dentro de cada clúster individual.

Cálculo paso a paso

Consideremos un conjunto de datos que ha sido particionado en k clústeres. El cálculo del índice de Davies-Bouldin se desarrolla a través de las siguientes etapas:

1. **Calcular el centroide de cada clúster**

Para cada clúster Ci, se determina su centroide μi, que representa el punto medio de todos los puntos pertenecientes a dicho clúster. El cálculo se realiza mediante la siguiente fórmula:

$$\mu i = \frac{1}{|Ci|} \sum_{x \in Ci} x$$

Donde:

- x representa cada uno de los puntos contenidos en el clúster Ci.
- |Ci| denota el número total de puntos asignados al clúster i.

2. Calcular la dispersión de cada clúster

La dispersión Si de un clúster Ci mide el grado en que los puntos individuales se encuentran extendidos o alejados con respecto al centroide de su clúster. Se calcula como el promedio de las distancias de cada punto del clúster a su centroide:

$$\mu i = \frac{1}{|Ci|} x \in Ci \sum x$$

Aquí, ‖x−μi‖ representa la distancia entre el punto x y el centroide μi, siendo comúnmente la distancia euclidiana. Un valor pequeño de Si indica un clúster más compacto.

3. Calcular las distancias entre cada par de centroides

Para cada par de clústeres distintos (Ci,Cj), se calcula la distancia Mi,j entre sus respectivos centroides μi y μj:

$$Mi,j = \| \mu i - \mu j \|$$

Esta distancia, generalmente euclidiana, proporciona una medida de la separación entre los clústeres i y j.

4. Calcular el "índice de similitud" entre clústeres

Para cada par de clústeres (i,j) donde i=j, se calcula un valor Ri,j que cuantifica la similitud entre ellos, considerando tanto su dispersión interna como su separación:

$$Ri,j = \frac{Si + Sj}{Mi,j}$$

Un valor bajo de Ri,j sugiere que los clústeres i y j son bien separados (gran Mi,j) y/o compactos (pequeños Si y Sj).

5. Obtener la peor similitud para cada clúster

Para cada clúster i, se identifica el clúster que presenta la mayor similitud Ri,j. Este valor máximo, denotado como Ri, representa la "peor" relación de similitud del clúster i con cualquier otro clúster, indicando el clúster más cercano o menos distinguible desde la perspectiva de i:

6. Calcular el índice Davies-Bouldin final

Finalmente, el índice de Davies-Bouldin (DBI) se obtiene calculando el promedio de los valores Ri para todos los clústeres:

$$DBI = \frac{1}{k}\sum_{i=1}^{k} R$$

Índice de Calinski-Harabasz

El índice de Calinski-Harabasz busca cuantificar la bondad de una solución de agrupamiento evaluando dos aspectos cruciales simultáneamente:

1. **Compacidad intraclúster (Within-cluster compactness):** Queremos que los puntos dentro de cada clúster sean lo más similares posible entre sí. Esto se traduce en que estén agrupados de forma densa alrededor del centroide de su clúster, minimizando la dispersión interna. Imagina un grupo de amigos que siempre se mantienen cerca en una fiesta.
2. **Separación interclúster (Between-cluster separation):** Deseamos que los diferentes clústeres estén bien diferenciados y alejados unos de otros en el espacio de datos. Esto significa que los centroides de los distintos clústeres deben estar lo más distantes posible. Volviendo a la analogía de la fiesta, queremos que los diferentes grupos de amigos estén en distintas áreas del salón, sin mezclarse demasiado.

Un buen algoritmo de agrupamiento logrará un equilibrio donde los clústeres son internamente cohesivos y externamente distintos. El índice CH nos proporciona un valor numérico que refleja este equilibrio. Un valor alto del índice CH sugiere una estructura de agrupamiento más definida.

Fórmula general del índice CH (con desglose de términos)

$$CH = \frac{Tr(Bk)}{Tr(Wk)} * \frac{n-k}{k-1}$$

Analicemos cada componente de esta fórmula:

- **n:** El número total de puntos u observaciones en nuestro conjunto de datos. Este es un valor fijo para un dataset dado.
- **k:** El número de clústeres que hemos generado con nuestro algoritmo de agrupamiento. Este valor puede variar dependiendo del algoritmo o de los

parámetros que hayamos utilizado. El índice CH se utiliza a menudo para comparar diferentes valores de k y ayudar a determinar el número óptimo de clústeres.

- **Tr(Bk) (Varianza entre clústeres):** Este término mide la dispersión de los clústeres *entre sí*. Cuanto mayor sea este valor, más separados estarán los clústeres. Se calcula sumando las distancias cuadradas ponderadas entre el centroide de cada clúster y el centroide global de todos los datos. La ponderación se realiza por el número de puntos en cada clúster.
- **Tr(Wk) (Varianza dentro de los clústeres):** Este término mide la dispersión de los puntos *dentro* de cada clúster. Cuanto menor sea este valor, más compactos serán los clústeres. Se calcula sumando las distancias cuadradas entre cada punto individual y el centroide del clúster al que pertenece.
- **k−1n−k (Factor de ajuste):** Este factor es un grado de libertad que ajusta el valor del índice CH en función del número de puntos (n) y el número de clústeres (k). Su propósito principal es penalizar la adición de más clústeres. Si simplemente tuviéramos el ratio Tr(Wk)Tr(Bk), tenderíamos a obtener un valor más alto al aumentar el número de clústeres, incluso si esos clústeres no son significativos. El factor de ajuste ayuda a mitigar este efecto. Observa que si k=1, el denominador se vuelve cero, lo que indica que el índice no está definido para un solo clúster.

Índices de Validación Externa

En el campo de la minería de datos, específicamente en el análisis de agrupamiento (*clustering*), uno de los principales desafíos es evaluar la calidad de las agrupaciones generadas por un algoritmo. Para ello, se utilizan técnicas de validación que permiten determinar qué tan adecuada es una partición de datos en clústeres. Estas técnicas se dividen en dos grandes categorías: **validación interna** y **validación externa**.

La **validación externa** se basa en la comparación de la agrupación obtenida por el algoritmo con una partición de referencia previamente conocida, también llamada **"ground truth"** o verdad del terreno. Esta partición de referencia puede provenir, por ejemplo, de etiquetas reales en un conjunto de datos etiquetado o de una clasificación previamente establecida por expertos. A diferencia de los índices internos, que solo consideran la estructura de los datos y la distribución de los clústeres, los índices externos permiten evaluar cuán bien una solución de agrupamiento reproduce una clasificación real esperada.

Índice de Rand Statistic

El **Índice de Rand** (propuesto por William M. Rand en 1971) es una medida que evalúa **la similitud entre dos particiones diferentes de un mismo conjunto de datos**. Se utiliza principalmente para comparar:

- La agrupación obtenida por un algoritmo.
- Una agrupación de referencia conocida .

El índice se basa en contar **pares de objetos** y evaluar **cuántas veces coinciden o no** en pertenecer al mismo clúster en ambas particiones.

¿Qué mide exactamente?

El Rand Index (RI) mide **la proporción de decisiones correctas** (es decir, pares que están juntos o separados en ambas particiones) sobre el total de pares posibles.

$$RI = \frac{a+b}{a+b+c+d}$$

Donde:

- a: número de pares de elementos que están en **el mismo clúster en ambas particiones**.
- b: número de pares de elementos que están en **diferentes clústeres en ambas particiones**.
- c: número de pares que están en **el mismo clúster en la partición real pero en diferentes clústeres en la partición generada**.
- d: número de pares que están en **diferentes clústeres en la partición real pero en el mismo clúster en la partición generada**.

El valor del RI está entre **0** (sin coincidencias) y **1** (coincidencia perfecta).

Limitación y mejora: Rand Index Ajustado (ARI)

Una gran **limitación del RI original** es que **no corrige el azar**. Es decir, incluso agrupaciones aleatorias pueden producir valores relativamente altos.

Por eso, se creó el **Adjusted Rand Index (ARI)**, que **ajusta el RI según la probabilidad de coincidencia aleatoria**. El ARI puede tomar valores negativos, y su valor máximo sigue siendo 1 (mejor coincidencia posible).

Indice de similitud de Jaccard

El **Coeficiente de Jaccard**, también conocido como *índice de similitud de Jaccard*, es una métrica utilizada para comparar la **similitud entre dos conjuntos**. En el contexto de minería de datos y validación externa de agrupamiento, se utiliza para comparar:

- La agrupación producida por un algoritmo de *clustering*.
- Una partición de referencia conocida (por ejemplo, las etiquetas reales o "ground truth").

Se enfoca **exclusivamente en los pares de elementos que son agrupados juntos**, lo que lo convierte en una métrica más estricta que otras como el Rand Index.

Fórmula del Coeficiente de Jaccard

La fórmula es:

$$J = \frac{a}{a + b + c}$$

Donde:

- a: Número de pares de elementos que están en el **mismo clúster en ambas particiones**.
- b: Número de pares que están en el **mismo clúster en la partición real pero en diferentes clústeres en la partición generada**.
- c: Número de pares que están en el **mismo clúster en la partición generada pero en diferentes clústeres en la partición real**.

Es decir, **no considera los pares correctamente separados** (caso ddd, como sí lo hace el Rand Index). Esto lo hace más estricto.

¿Qué mide?

El Coeficiente de Jaccard mide la **proporción de coincidencias positivas** (pares que están correctamente agrupados juntos) **respecto al total de pares que fueron agrupados juntos en al menos una de las particiones**.

Sus valores posibles van de:

- **0**: No hay coincidencia (los elementos agrupados juntos en una partición están siempre separados en la otra).
- **1**: Coincidencia perfecta (todos los elementos agrupados juntos en una partición también lo están en la otra).

Indice de Fowlkes & Mallows

El **índice de Fowlkes & Mallows (FM)** es una métrica desarrollada por **E. B. Fowlkes y C. L. Mallows** en 1983. Se utiliza para **comparar la similitud entre una partición generada por un algoritmo de agrupamiento y una partición de referencia (ground truth)**.

Este índice está basado en la idea de que si dos elementos están agrupados juntos en una partición, también deberían estar juntos en la otra. Es decir, **solo considera los pares que han sido agrupados**, y mide cuán "precisos" y "puros" son esos grupos.

¿Qué mide?

El índice de Fowlkes & Mallows es esencialmente la **media geométrica** entre:

- La **precisión** de los pares agrupados (proporción de pares agrupados correctamente).
- La **recuperación** (proporción de pares verdaderamente relacionados que fueron agrupados).
-

Fórmula general

$$FM = \sqrt{\frac{a}{a+b} * \frac{a}{a+c}}$$

Donde:

- a: número de pares de elementos que están en el **mismo clúster en ambas particiones**.
- b: número de pares que están **juntos en la partición generada pero no en la referencia**.
- c: número de pares que están **juntos en la referencia pero no en la partición generada**.

Explicación de los componentes

- aa+b\frac{a}{a + b}a+ba: **Precisión** → proporción de pares que el algoritmo agrupó y que **realmente pertenecían juntos**.
- aa+c\frac{a}{a + c}a+ca: **Recuperación** → proporción de pares correctos (según la verdad real) que **fueron recuperados** por el algoritmo.

El FM combina ambas dimensiones de forma equilibrada usando una media geométrica.

Ventajas del FM

- Tiene una interpretación intuitiva: combina precisión y recuperación.
- Es simétrico.
- Es menos sensible a grandes cantidades de pares correctamente separados (caso que puede sesgar otras métricas).

Naive Bayes

En el ámbito del aprendizaje automático, una de las tareas más frecuentes es la clasificación supervisada, que se usa para anticipar etiquetas a partir de datos que ya han sido etiquetados. Uno de los métodos más simples pero notablemente efectivos para esta labor es el clasificador Naive Bayes. Este enfoque se basa en ideas de probabilidad y utiliza el teorema de Bayes, haciendo una suposición “ingenua” de que las características del conjunto de datos son independientes entre sí. Debido a su bajo costo de procesamiento y su excelente rendimiento, especialmente en la categorización de texto, se ha transformado en un recurso esencial para investigadores y creadores de sistemas inteligentes.

Flujo de trabajo de la clasificación:

Recopilación de datos: Se recolectan datos clasificados que incluyen ejemplos junto a sus categorías correspondientes.

Preprocesamiento: Se organizan y depuran los datos (eliminación de interferencias, conversión de texto).

División del conjunto: El conjunto de datos se divide en grupo con el cual se entrena y otro con el que se prueba.

Entrenamiento del modelo: Se emplea el grupo de entrenamiento para optimizar el modelo (como Naive Bayes).

Evaluación: Se mide como funciona el modelo mediante indicadores como la exactitud, recuperación y F1-Score.

Predicción: Después de la validación, el modelo se emplea para categorizar nuevos datos que no se han visto antes.

¿Cómo funciona el clasificador Naive Bayes?

Se aplica el teorema de Bayes.

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

Figura 1.1. *Teorema de Bayes, base fundamental del clasificador Naive Bayes para calcular la probabilidad posterior de una clase dada una observación.*

Representa la posibilidad posterior ***(Y|X)***, esto quiere decir la probabilidad de que suceda un evento ***Y***, sabiendo que ya ha sucedido el evento ***X***. Este número se determina al multiplicar:

P(X|Y): La verosimilitud, o la probabilidad de observar ***X*** si se sabe que ***Y*** es verdadero.

P(Y): La probabilidad previa de que ocurra ***Y.***

Y dividiendo ese producto entre

P(X): La probabilidad total de observar ***X***, independientemente de ***Y.***

Este teorema fundamenta el clasificador de Naive Bayes, que busca determinar la clase más probable para la observación específica, teniendo en cuenta las probabilidades anteriores y condicionadas.

Redes Neuronales Artificiales

Las redes neuronales artificiales, conocidas como RNA, son modelos informáticos que se basan en la forma en que opera el cerebro humano. Estas imitan la habilidad de aprender y generalizar de las neuronas naturales utilizando sistemas de procesamiento que trabajan de manera distribuida y simultánea. Estas redes resultan muy efectivas en situaciones donde los métodos convencionales no funcionan bien, como identificar patrones, clasificar información y hacer predicciones.

Características de las Redes Neuronales Artificiales

Las RNA contienen diferentes características:

- **Su aprendizaje es adaptable:** Tienen la capacidad de modificar sus ajustes internos (pesos sinápticos) según la información de entrada y salida a lo largo del entrenamiento.
- **Generalización:** Después de haber sido entrenadas, pueden ofrecer respuestas precisas a datos que no han sido vistos con anterioridad, siempre y cuando estos sean parecidos a las del conjunto de entrenamiento.
- **Procesamiento paralelo distribuido:** Los datos se manejan a la vez en varias unidades, lo que proporciona una eficacia y resistencia a errores.
- **Tolerancia a Ruido y Errores:** Gracias a su ya mencionado diseño distribuido, las Redes Neuronales Artificiales continúan operando de manera adecuada a pesar de que algunas de sus partes fallen o que haya ruido en las entradas.
- **Capacidad de autoorganización:** Ciertas redes tienen la capacidad de establecer su propia estructura interna sin necesidad de ayuda externa, identificando patrones y conexiones en la información de manera autónoma.

Estructura Básica de una Red Neuronal:

La configuración fundamental de una red neuronal artificial (RNA) se basa en cómo opera el cerebro humano y está formada por capas de elementos de procesamiento conocidos como neuronas artificiales. Estas neuronas se despliegan en tres clases de capas principales:

- **Capa de entrada:** Esta es la capa inicial de la red y tiene la responsabilidad de aceptar datos entrantes. Cada nodo en capa simboliza una característica del conjunto de datos y envía esta información a las siguientes capas.
- **Capas ocultas:** Se trata de una o más capas que se encuentran entre la entrada y la salida. Cada neurona en estas capas recibe información de la capa previa, la procesa a través de una función de activación que no es lineal y envía el resultado a la siguiente capa. Estas capas tienen la tarea de aprender y mostrar patrones complejos presentes en los datos.
- **Capa de salida:** Esta es la última capa que genera el resultado de la red. La cantidad de neuronas en estas capas varía según el tipo de trabajo.

Cada enlace entre neuronas posee un valor que define el impacto de una neurona en otra. A lo largo del proceso de entrenamiento de la red, estos valores se modifican para reducir la diferencia entre la salida esperada y la salida real. Esta modificación se lleva a cabo a través de métodos de aprendizaje, como el algoritmo de retropropagación del error.

Capítulo 2: Herramientas y Técnicas Utilizadas

2.1. Introducción

En este capítulo se detallan las herramientas y técnicas utilizadas para implementar los algoritmos de clustering en Python. Se explican las bibliotecas clave, su función en el análisis de datos y cómo configurar correctamente los parámetros de cada algoritmo para obtener resultados óptimos. Además, se aborda el preprocesamiento de datos, una etapa esencial para garantizar la calidad de los agrupamientos.

2.2. Descripción del Software Utilizado

2.2.1. Python y Bibliotecas Clave

Python es el lenguaje de programación elegido por su versatilidad, facilidad de uso y amplio ecosistema de bibliotecas para ciencia de datos. A continuación, se describen las principales bibliotecas utilizadas y su propósito:

NumPy:

- **Función**: Proporciona estructuras de datos eficientes (como arrays multidimensionales) y operaciones matemáticas avanzadas.
- **Uso en clustering**: Permite manipular matrices de datos y calcular distancias entre puntos, esencial para algoritmos como K-Means y DBSCAN.

Pandas:

- **Función**: Facilita la manipulación y análisis de datos estructurados mediante DataFrames.
- **Uso en clustering**: Carga, filtra y prepara los datos antes de aplicar los algoritmos.

Matplotlib y Seaborn:

- **Función**: Generan visualizaciones para explorar y validar los resultados del clustering.
- **Uso en clustering**: Gráficos de dispersión, dendrogramas (para clustering jerárquico) y métricas como el método del codo.

scikit-learn (sklearn):

- **Función**: Biblioteca principal para machine learning en Python, con implementaciones optimizadas de algoritmos de clustering.
- **Uso en clustering**: Contiene clases como KMeans, DBSCAN y AgglomerativeClustering.

SciPy:

- **Función**: Proporciona herramientas avanzadas para computación científica.
- **Uso en clustering**: Cálculo de matrices de distancia (útil en clustering jerárquico).

LabelEncoder:

- **Función**: Transforma etiquetas categóricas en valores numéricos.

- **Uso en clustering**: Permite codificar las etiquetas verdaderas para compararlas con los resultados de clustering y evaluar modelos supervisados.

train_test_split:

- **Función**: Divide el dataset en conjuntos de entrenamiento y prueba.
- **Uso en clustering**: Ayuda a validar clasificadores entrenados con etiquetas generadas por clustering comparándolos con etiquetas reales.

PCA :

- **Función**: Reduce la dimensionalidad de los datos para facilitar su visualización.
- **Uso en clustering**: Permite proyectar los datos en 2D para visualizar las agrupaciones obtenidas.

GaussianNB:

- **Función**: Clasificador basado en la distribución normal (naive Bayes).
- **Uso en clustering**: Se entrena con etiquetas verdaderas o generadas por clustering para evaluar la calidad de los agrupamientos.

MLPClassifier:

- **Función**: Implementa una red neuronal multicapa para clasificación supervisada.
- **Uso en clustering**: Evalúa la capacidad de generalización del etiquetado generado por clustering al entrenar con dichas etiquetas.

accuracy_score y f1_score:

- **Función**: Miden el rendimiento de un modelo de clasificación.
- **Uso en clustering**: Evalúan cuán similares son las etiquetas generadas por clustering respecto a las reales.

scipy.io.arff

- **Función**: Carga archivos en formato ARFF (Attribute-Relation File Format).
- **Uso en clustering**: Permite importar datasets anotados con etiquetas, útiles para comparar con resultados de clustering no supervisado.

2.2.2. Instalación y Configuración

Todas las bibliotecas mencionadas pueden instalarse usando pip, el gestor de paquetes de Python.

Se recomienda trabajar en un entorno virtual (venv o conda) para evitar conflictos entre versiones.

2.3. Configuración de Algoritmos de Clustering

2.3.1. K-Means

Clase en scikit-learn: sklearn.cluster.KMeans

Parámetros clave y su explicación:

1. n_clusters (int):
    - **Qué es**: Número de clusters a formar.
    - **Cómo determinarlo**:
        - **Método del codo**: Graficar la suma de cuadrados intra-cluster (inercia) para diferentes valores de *k* y elegir el punto donde la disminución se estabiliza.
        - **Silhouette Score**: Mide la cohesión y separación de los clusters (valores cercanos a 1 indican mejor agrupamiento).
2. init (str o array):
    - **Qué es**: Método de inicialización de centroides.
    - **Opciones**:
        - "k-means++" (recomendado): Selecciona centroides iniciales de forma inteligente para acelerar la convergencia.
        - "random": Inicialización aleatoria (menos eficiente).
3. max_iter (int):
    - **Qué es**: Número máximo de iteraciones por ejecución.
    - **Valor típico**: 300 (suficiente para la mayoría de casos).
4. n_init (int):
    - **Qué es**: Número de veces que se ejecuta el algoritmo con diferentes inicializaciones.
    - **Recomendación**: 10 (para evitar mínimos locales).

2.3.2. DBSCAN

Clase en scikit-learn: sklearn.cluster.DBSCAN

Parámetros clave y su explicación:

1. eps (float):
    - **Qué es**: Distancia máxima entre dos puntos para ser considerados vecinos.
    - **Cómo determinarlo**:

        - Analizar la distribución de distancias con KNN (K-Nearest Neighbors) y elegir un valor donde la curva muestra un "codo".

2. min_samples (int):
    - **Qué es**: Número mínimo de puntos para formar un cluster.
    - **Recomendación**:
        - Valores bajos (3-5) para datos con ruido.
        - Valores altos (10+) para datos más densos.

2.3.3. Clustering Jerárquico Aglomerativo

Clase en scikit-learn: sklearn.cluster.AgglomerativeClustering

Parámetros clave y su explicación:

1. n_clusters (int):
    - **Qué es**: Número de clusters finales.
    - **Cómo determinarlo**:
        - Analizar un dendrograma (scipy.cluster.hierarchy.dendrogram) y cortar donde la distancia entre clusters aumenta significativamente.
2. linkage (str):
    - **Qué es**: Criterio para fusionar clusters.
    - **Opciones comunes**:
        - "ward" (recomendado para datos homogéneos): Minimiza la varianza intra-cluster.
        - "complete": Usa la distancia máxima entre puntos.
        - "average": Usa la distancia promedio.

2.4. Preprocesamiento de Datos

Pasos esenciales antes de aplicar clustering:

1. **Normalización/Estandarización**:
    - **Por qué**: Algoritmos como K-Means son sensibles a la escala de los datos.
    - **Herramienta**: StandardScaler de scikit-learn.
1. **Manejo de valores nulos y outliers**:
    - **Estrategias**:
        - Eliminar filas con nulos (df.dropna()).

        - Imputar valores (media, mediana).
        - Usar DBSCAN para detectar outliers automáticamente.

2.5. Configuración de algoritmo de clustering

2.5.1. Naive-bayes

- **Clase en scikit-learn:** sklearn.naive_bayes.GaussianNB
    - **var_smoothing (float):**
    - **Qué es**: Parámetro de suavizado para evitar divisiones por cero.
    - **Valor típico:** 1e-9 (por defecto).
    - **Consideraciones**: Generalmente no se modifica, salvo en casos de datos muy ruidosos o mal distribuidos.

2.5.2. Red Neuronal

Clase en scikit-learn: sklearn.neural_network.MLPClassifier

1. **hidden_layer_sizes (tuple):**
    - **Qué es**: Número y tamaño de las capas ocultas.
    - **Ejemplo**: (50,) indica una sola capa con 50 neuronas.
    - **Recomendación**: Ajustar según la complejidad del problema (más capas/neuronas para tareas más complejas).
2. **max_iter (int):**
    - **Qué es**: Número máximo de iteraciones durante el entrenamiento.
    - **Valor típico**: 200–500.
    - **Recomendación**: Aumentar si la red no converge (lo indica una advertencia en scikit-learn).
3. **activation (str):**
    - Qué es: Función de activación de las neuronas.
    - Opciones comunes:
        - "relu" (recomendado): Buen rendimiento en la mayoría de los casos.
        - "tanh" y "logistic": Útiles en redes más pequeñas o tareas específicas.
4. **solver (str):**
    - Qué es: Algoritmo de optimización.
        - Opciones:
            - "adam" (por defecto): Buen rendimiento general.
            - "sgd": Mejor control con tasas de aprendizaje manuales.
            - "lbfgs": Útil en datasets pequeños.

Capítulo 3 – Resultados

En este capítulo se presentan los resultados obtenidos tras aplicar diversos algoritmos de agrupamiento no supervisado sobre el conjunto de datos *Dry Beans Dataset*, que contiene 13,611 muestras caracterizadas mediante 16 atributos geométricos y morfológicos de granos de frijol. Para todos los métodos, se excluyó deliberadamente la etiqueta de clase original con el objetivo de obtener particiones derivadas únicamente de la estructura de los datos.

Los métodos aplicados fueron K-Means, agrupamiento jerárquico aglomerativo y DBSCAN. A cada una de las particiones generadas se les calcularon los índices de validez interna Calinski-Harabasz y Davies-Bouldin. Finalmente, se entrenó un modelo de red neuronal supervisada con el fin de comparar el rendimiento de clasificación contra la partición seleccionada como la mejor, así como contra las etiquetas reales.

3.1 Descripción de los datos

El Dry Bean Dataset es un conjunto de datos de seis especies de frijoles secos, obtenido a partir de imágenes de frijoles turcos mediante procesamiento digital. Cada instancia representa un frijol individual y contiene características geométricas calculadas automáticamente.

Dimensiones del Dataset

- Total de instancias (frijoles): 13,611
- Número de atributos: 16 características numéricas + 1 etiqueta de clase (tipo de frijol)

Atributos del Dataset

Todos los atributos excepto la clase son numéricos continuos y representan características derivadas del contorno y forma del frijol. A continuación se describe cada uno:

Atributo	Descripción
Area	Número total de píxeles del objeto (área del frijol).
Perimeter	Longitud del contorno del frijol.
MajorAxisLength	Longitud del eje mayor de la elipse ajustada al frijol.
MinorAxisLength	Longitud del eje menor de la elipse ajustada.
AspectRation	Relación entre el eje mayor y el eje menor (alargamiento).

Eccentricity	Medida de qué tan alargada está la forma (cercana a 1 = más alargada).
ConvexArea	Área del polígono convexo más pequeño que rodea al frijol.
EquivDiameter	Diámetro del círculo con la misma área que el frijol.
Extent	Proporción entre el área del frijol y el área de su caja delimitadora.
Solidity	Relación entre el área y el área convexa (cercana a 1 = menos irregular).
Roundness	Qué tan circular es la forma.
Compactness	Medida de qué tan densa o “compacta” es la figura.
ShapeFactor1	Forma derivada del perímetro y área.
ShapeFactor2	Otra medida derivada de forma.
ShapeFactor3	Basada en el radio del área al cuadrado.
ShapeFactor4	Medida alternativa de circularidad.
Class	Representa el tipo de frijol seco.

Tabla 3.1 – Descripción de todos los atributos encontrados en la dataset Dry Bean Dataset

Etiqueta de clase (Class)

Representa el tipo de frijol seco. Hay 7 clases:

Clase	Frecuencia	Porcentaje
DERMASON	3,546	26.1%

SIRA	2,636	19.4%
SEKER	2,027	14.9%
HOROZ	1,928	14.2%
CALI	1,630	12.0%
BARBUNYA	1,322	9.7%
BOMBAY	522	3.8%

Tabla 3.2 – Representación de las 7 clases contenida en la dataset Dry Bean Dataset

3.2 Preprocesamiento de los datos

Se removió la columna 'Class' del conjunto de datos, ya que esta corresponde a la variable objetivo (etiquetas de agrupamiento/clasificación).

El objetivo era trabajar únicamente con las características (features) para prepararlas antes de aplicar algoritmos de aprendizaje no supervisado (como clustering) o para separar el preprocesamiento de las etiquetas en aprendizaje supervisado.

Normalización estándar (StandardScaler)

Los datos se transformaron para que cada característica tuviera:

Media = 0 (centrado en cero).

Desviación estándar = 1 (misma escala).

Esto asegura que todas las variables tengan igual importancia en algoritmos sensibles a la magnitud de los datos (como K-Means, SVM o PCA).

Efecto en los datos
Sin 'Class': El dataset ahora contiene solo variables predictoras, listas para técnicas de agrupamiento (clustering) o reducción de dimensionalidad.

Normalizados: Las features ya no están en sus unidades originales (ej: píxeles, milímetros), sino en una escala comparable, evitando sesgos por diferencias de magnitud.

3.3 Aplicación del algoritmo K-Means

Se aplicó el algoritmo de K-Means al conjunto de datos preprocesado, considerando valores de *k* igual a 6, 7 y 8, con el fin de explorar la correspondencia entre el número real de clases (7) y las agrupaciones emergentes a partir de la estructura no etiquetada.

Para cada uno de los valores de *k*, se presentan a continuación:

- Una visualización 2D de las agrupaciones proyectadas mediante PCA.
- Una tabla con la distribución de muestras por grupo.
- Resultados de los índices de validez Calinski-Harabasz y Davies-Bouldin (resumidos más adelante).

k = 6

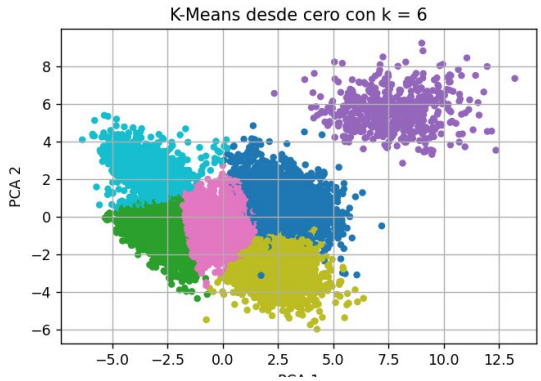


Figura 3.1 *– Proyección bidimensional mediante PCA de los grupos generados por K-Means con k=6. Cada color representa un grupo distinto.*

k = 7

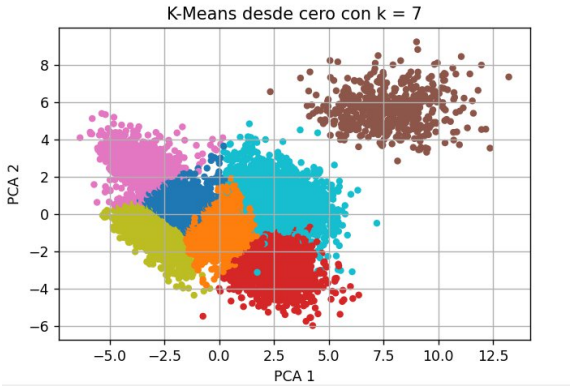


Figura 3.2 *– Proyección bidimensional mediante PCA de los grupos generados por K-Means con k=7. Cada color representa un grupo distinto.*

k = 8

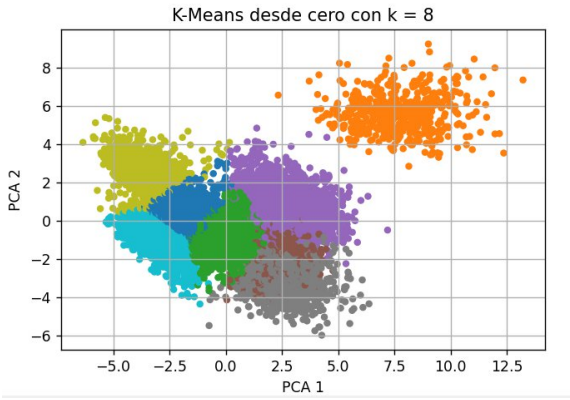


Figura 3.3 *– Proyección bidimensional mediante PCA de los grupos generados por K-Means con k=7. Cada color representa un grupo distinto.*

De la ejecución del algoritmo K-Means, se obtuvieron las siguientes distribuciones de instancias por grupo para cada número de particiones.

K	G0	G1	G2	G3	G4	G5	G6	G7
6	2773	3189	521	3143	1947	2038		
7	2002	2246	1884	521	1848	2339	2771	
8	2003	521	2202	2685	964	1068	1848	2320

Tabla 3.3 – Distribución del número de instancias por grupo para el algoritmo K-Means con k=6, k=7 y k=8.

Se calcularon los índices de validez para cada configuración:

No. de Particiones	Davies-Bouldin	Calinski-Harabasz
6	1.1386	7950.205
7	1.2105	7001.3383

8	1.2476	6589.2179

Tabla 3.4 – Resultados de los índices de Calinski-Harabasz y Davies-Bouldin para K-Means con distintas particiones.

3.4 Aplicación del algoritmo Jerárquico Aglomerativo

Se realizó el agrupamiento jerárquico con enfoque aglomerativo utilizando el mismo conjunto de datos y las mismas configuraciones de número de clusters: 6, 7 y 8.

En cada caso se generaron los siguientes resultados:

- Gráfica de los datos agrupados en 2D mediante PCA.
- Tabla de distribución de elementos por grupo.
- Resultados de los índices de validez Calinski-Harabasz y Davies-Bouldin (resumidos más adelante).

k = 6

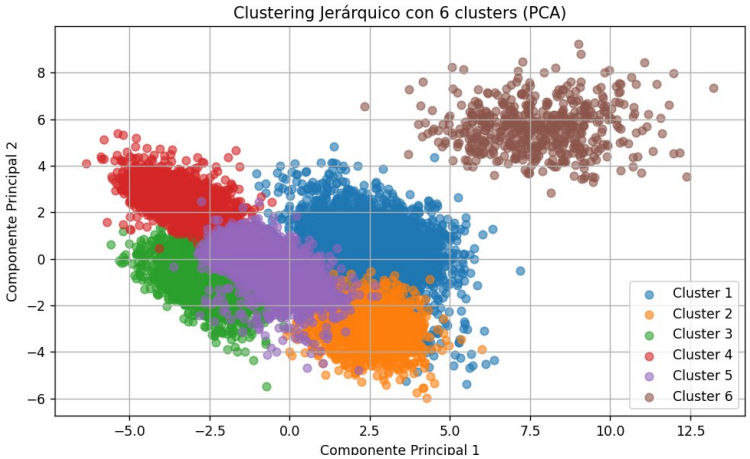


Figura 3.4 *– Proyección PCA de los grupos generados por el método jerárquico con k=6.*

k = 7

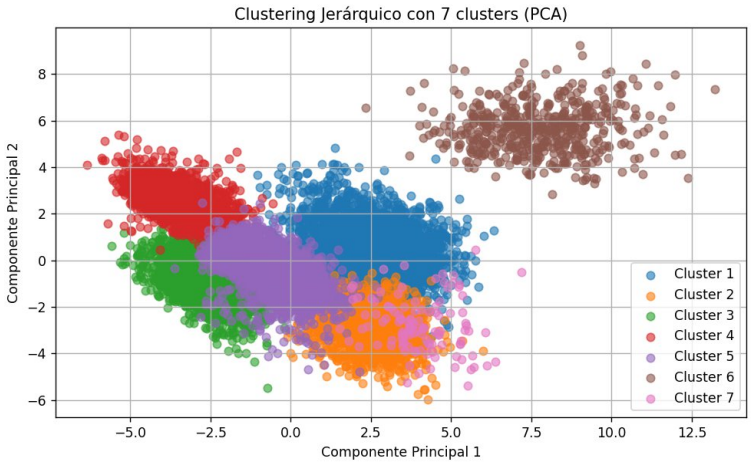


Figura 3.5 *– Proyección PCA de los grupos generados por el método jerárquico con k=7*

k = 8

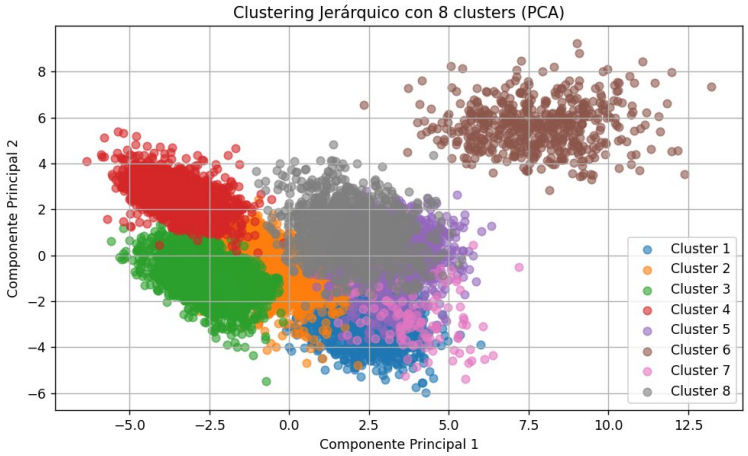


Figura 3.6 *– Proyección PCA de los grupos generados por el método jerárquico con k=8*

De la ejecución del algoritmo jerárquico aglomerativo, se obtuvieron las siguientes distribuciones de instancias por grupo para cada número de particiones.

K	*G0*	*G1*	*G2*	*G3*	*G4*	*G5*	*G6*	*G7*
6	*3153*	*1677*	*3063*	*1918*	*3278*	*522*		
7	*3036*	*1677*	*3063*	*1918*	*3278*	*522*	*117*	

8	*1760*	*1677*	*3063*	*1918*	*3278*	*522*	*117*	*1276*

Tabla 3.5 *– Distribución del número de instancias por grupo para el algoritmo jerárquico con k=6, k=7 y k=8.*

Los índices de validez aplicados sobre cada partición generada fueron los siguientes:

No. de Particiones	*Davies-Bouldin*	*Calinski-Harabasz*
6	*1.1875*	*7396.8843*
7	*1.1064*	*6981.2745*
8	*1.2508*	*6626.5772*

Tabla 3.6 – Resultados de los índices de Calinski-Harabasz y Davies-Bouldin para agrupamientos jerárquicos.

3.5 Aplicación de DBSCAN

Debido a la naturaleza del algoritmo DBSCAN, se llevó a cabo una exploración extensa de combinaciones de los parámetros epsilon (eps) y mínimo número de puntos vecinos (minPts), con el fin de identificar configuraciones que produjeran agrupamientos con poca cantidad de ruido y una cantidad de grupos cercana a 7.

Se realizó una bitácora de aproximadamente 50 pruebas distintas, registrando los valores utilizados y las métricas observadas en cada intento. **Esta bitácora se incluye como anexo.**

A continuación, se muestran las tres configuraciones seleccionadas por su distribución equilibrada, bajo ruido y número de grupos aceptable:

Configuración 1
eps = 1.25, minPts = 4

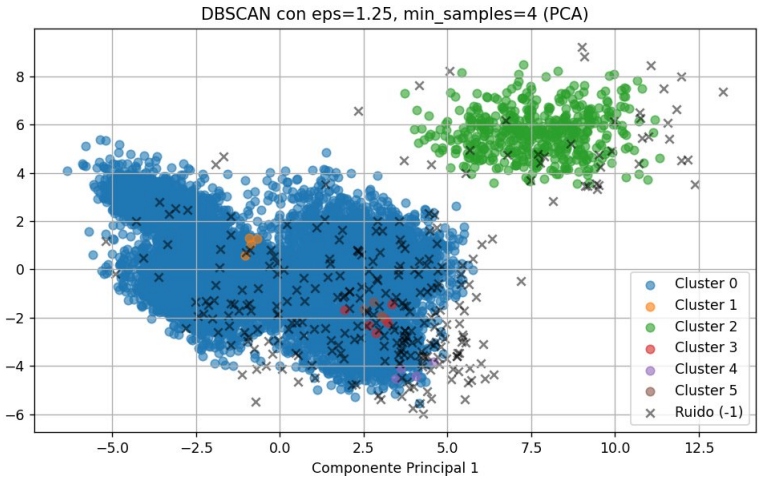


Figura 3.7 – Proyección PCA de los grupos generados por DBSCAN – Configuración 1.

Configuración 2
eps = 1.45, minPts = 3

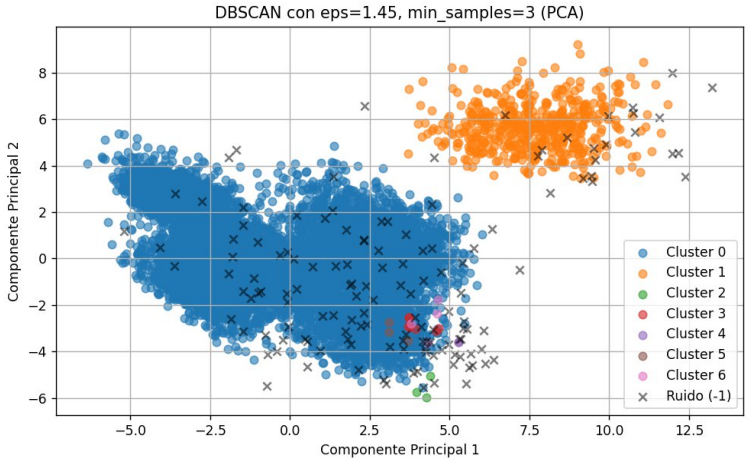


Figura 3.8 – Proyección PCA de los grupos generados por DBSCAN – Configuración 2.

Configuración 3
eps = 0.97, minPts = 5

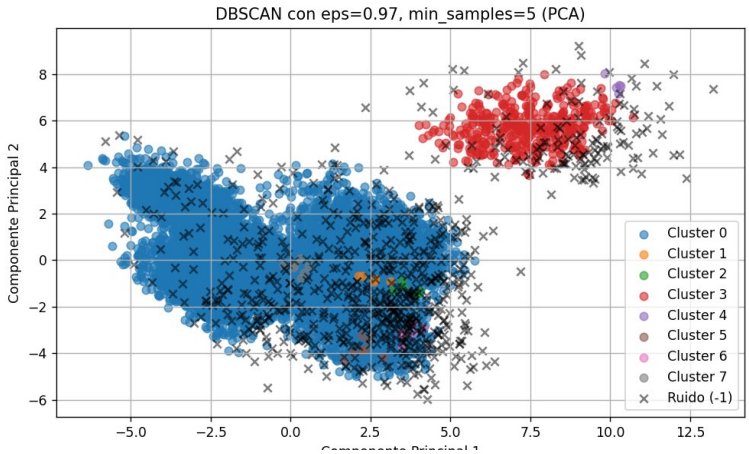


Figura 3.9 *– Proyección PCA de los grupos generados por DBSCAN – Configuración 3.*

De las ejecuciones de DBSCAN, se obtuvieron las siguientes distribuciones de instancias por grupo para cada una de las configuraciones seleccionadas:

eps	*min_pts*	*# de grupos*	*Elementos ruido*	*Distribución de los grupos*
0.97	*5*	*8*	*789*	*Ruido: 789, G0: 12406, G1: 5, G2: 5, G3: 381, G4: 5, G5: 7, G6: 6, G7: 7*
1.25	*4*	*6*	*288*	*Ruido: 288, G0: 12823, G1: 4, G2: 482, G3: 6, G4: 4, G5: 4*
1.45	*3*	*7*	*140*	*Ruido: 140, G0: 12952, G1: 500, G2: 3, G3: 7, G4: 3, G5: 3, G6: 3*

Tabla 3.7 *– Distribución del número de instancias por grupo para DBSCAN con tres configuraciones diferentes.*

A continuación se presentan los resultados de los índices de validez para cada configuración:

Configuración	*Davies-Bouldin*	*Calinski-Harabasz*
eps = 1.25, minPts=4	*0.8048*	*828.0468*
eps = 1.45, minPts=3	*0.7029*	*711.5532*
eps = 0.97, minPts=5	*0.8048*	*828.0468*

Tabla 3.8 *– Resultados de los índices de Calinski-Harabasz y Davies-Bouldin para agrupamientos generados por DBSCAN.*

3.6 Evaluación mediante índices de validez

A las nueve particiones obtenidas (3 de cada algoritmo) se les calcularon los índices de Calinski-Harabasz y Davies-Bouldin, con el objetivo de estimar la calidad interna de las agrupaciones sin referencia a las clases reales.

A continuación se presenta una tabla comparativa con los valores obtenidos para cada configuración:

Algoritmo	*Configuración*	*Davies-Bouldin*	*Calinski-Harabasz*
Jerárquico	*k = 6*	*1.1875*	*7396.8843*
Jerárquico	*k = 7*	*1.1064*	*6981.2745*
Jerárquico	*k = 8*	*1.2508*	*6626.5772*

K-means	***k = 6***	***1.1386***	***7950.205***
K-means	*k = 7*	*1.2105*	*7001.3383*
K-means	*k = 8*	*1.2476*	*6589.2179*
DBSCAN	*eps = 1.25, minPts = 4*	*0.8048*	*828.0468*
DBSCAN	*eps = 1.45, minPts = 3*	*0.7029*	*711.5532*
DBSCAN	*eps = 0.97, minPts = 5*	*0.8048*	*828.0468*

Tabla 3.9 *– Comparación de los índices de validez entre todas las particiones generadas.*

La mejor partición fue la obtenida con el algoritmo **K-means**, en su configuración de **k = 6**, ya que presentó un índice de Calinski-Harabasz de **7950.2050** y un índice de Davies-Bouldin de **1.1386**. Esto indica que los grupos generados presentan una buena separación entre sí y alta cohesión interna, lo cual refleja un agrupamiento de calidad sobre el conjunto de datos.

Dicha partición será considerada más adelante para pruebas experimentales de clasificación supervisada, tomándola como generación sintética de etiquetas para el conjunto de datos de frijoles secos.

3.7 Clasificación supervisada con etiquetas reales y sintéticas

Con el objetivo de evaluar la capacidad predictiva del conjunto de datos, se realizaron experimentos de clasificación supervisada utilizando dos enfoques distintos: un modelo basado en Naive-Bayes y un modelo de Red Neuronal Artificial. Ambos fueron aplicados en dos contextos distintos:

1. Usando las **etiquetas reales** del conjunto de datos como referencia.
2. Usando las **etiquetas sintéticas generadas** mediante el algoritmo K-Means con k=6, consideradas como una alternativa no supervisada.

Esta comparación permite analizar la calidad de los agrupamientos no supervisados desde una perspectiva supervisada, evaluando su utilidad como sustituto aproximado de las etiquetas reales.

Clasificación con etiquetas reales

En este primer experimento se entrenaron los modelos usando como objetivo la clase real de cada instancia. Previamente, los datos fueron normalizados y divididos en conjuntos de entrenamiento y prueba. Se evaluaron los modelos mediante las métricas **accuracy** y **F1-score**, además de visualizar la distribución de clases mediante PCA.

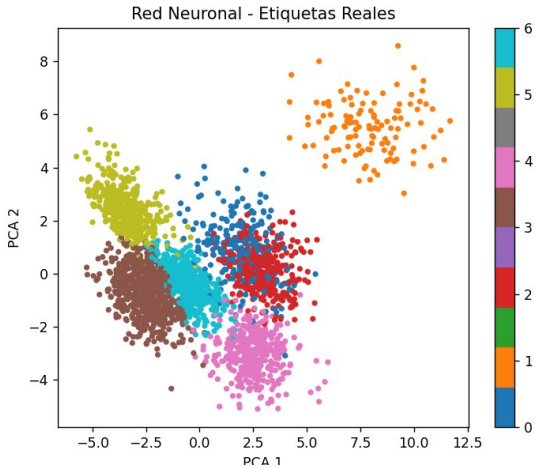


Figura 3.10 – Proyección PCA de la clasificación obtenida con etiquetas reales mediante red neuronal.

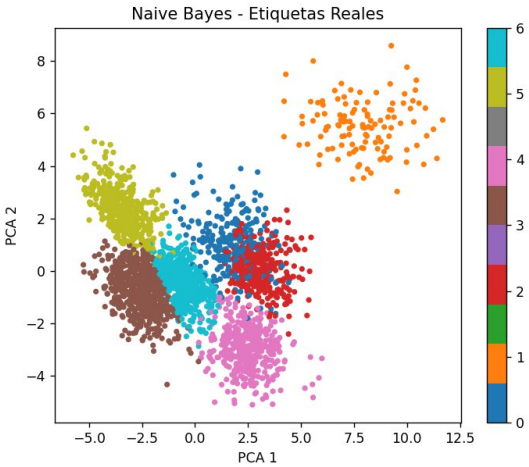


Figura 3.11 – Proyección PCA de la clasificación obtenida con etiquetas reales mediante Naive-Bayes.

Los resultados obtenidos fueron los siguientes:

- **Naive Bayes:**
  - Accuracy: 0.9038

    - F1 Score: 0.9115
- **Red Neuronal:**
    - Accuracy: 0.9357
    - F1 Score: 0.9461

Clasificación con etiquetas sintéticas (K-Means, k=6)

En este segundo experimento se utilizaron las etiquetas generadas por K-Means con k=6 como clases objetivo, bajo el supuesto de haber obtenido una partición óptima y representativa procesando los datos con algoritmos no supervisados. El proceso de entrenamiento y evaluación fue idéntico al anterior.

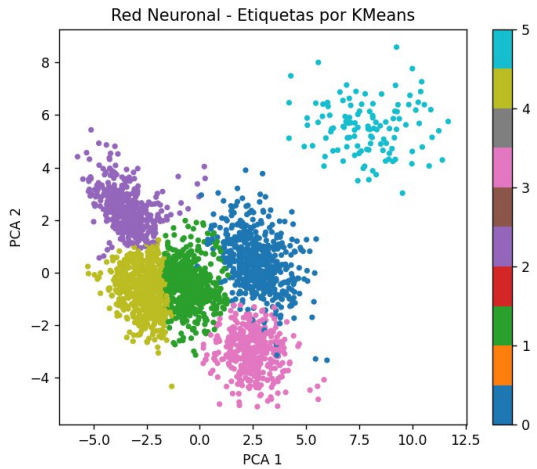


Figura 3.12 – Proyección PCA de la clasificación obtenida con etiquetas sintéticas mediante red neuronal.

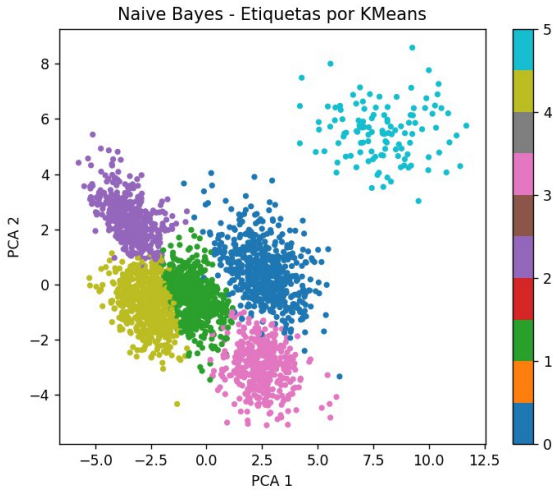


Figura 3.13 – Proyección PCA de la clasificación obtenida con etiquetas sintéticas mediante Naive-Bayes.

Los resultados obtenidos en este contexto fueron:

- **Naive Bayes:**
  - Accuracy: 0.9633
  - F1 Score: 0.9693
- **Red Neuronal:**
  - Accuracy: 0.9967
  - F1 Score: 0.9972

Comparación de resultados

A continuación se resumen los resultados obtenidos por ambos clasificadores en ambos escenarios, permitiendo una comparación directa entre el rendimiento con etiquetas reales y etiquetas generadas sintéticamente.

Método de Clasificación	Tipo de Etiqueta	Accuracy	F1 Score
Naive Bayes	Reales	0.9038	0.9115
Red Neuronal	Reales	0.9357	0.9461
Naive Bayes	K-Means (k=6)	0.9633	0.9693
Red Neuronal	K-Means (k=6)	0.9967	0.9972

Tabla 3.10 – Comparación del desempeño de los clasificadores con etiquetas reales y sintéticas (K-Means), evaluados por Accuracy y F1 Score.

Para facilitar la visualización de las diferencias, se presentan a continuación dos gráficas de barras con los valores de Accuracy y F1 Score respectivamente.

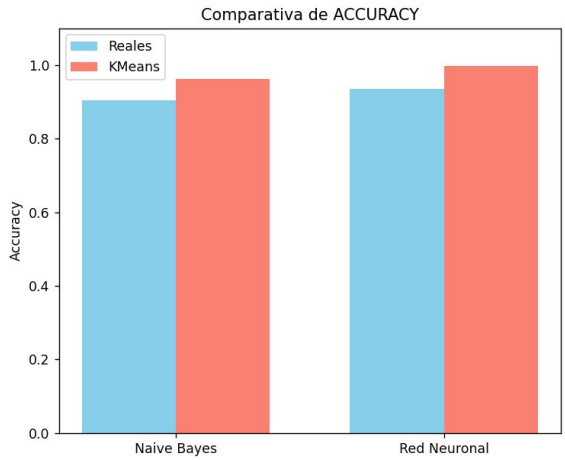


Figura 3.14 – Comparación gráfica de Accuracy para los distintos clasificadores y tipos de etiqueta.

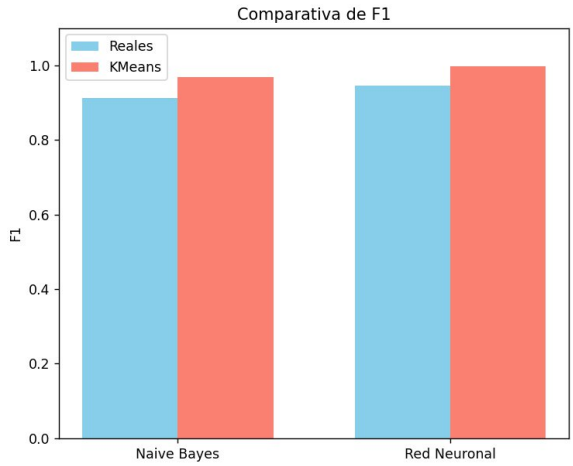


Figura 3.15 – Comparación gráfica de F1 Score para los distintos clasificadores y tipos de etiqueta.

Análisis de resultados

Los resultados muestran que, sorprendentemente, los clasificadores alcanzan mayor desempeño cuando se utilizan las etiquetas sintéticas generadas por K-Means con k=6, superando a las etiquetas reales tanto en Accuracy como en F1 Score. Esto podría explicarse por dos factores:

- El agrupamiento pudo haber simplificado la estructura del conjunto de datos, reduciendo la complejidad del problema.

- La reducción a 6 grupos en lugar de 7 tal vez evitó clases con alto solapamiento o desequilibrio, facilitando la tarea de clasificación.

Este hallazgo sugiere que, en contextos donde no se dispone de etiquetas reales, un buen agrupamiento no supervisado puede proporcionar información útil para tareas supervisadas, mejorando incluso la capacidad predictiva del modelo.

Conclusiones

En este análisis, se ha examinado cómo se pueden utilizar los métodos de minería de datos para explorar datos multidimensionales y obtener información de ellos. Se requería utilizar datos del mundo real que fueran realmente fáciles de observar, cuyo comportamiento en diferentes algoritmos de aprendizaje automático fuera lo suficientemente descriptivo.

El uso de técnicas como K-Means, el agrupamiento jerárquico y DBSCAN demostró que cada algoritmo proporciona una visión diferente acerca de la distribución interna de los datos. Modificar sus parámetros, analizar los resultados y cotejarlos no solo facilitó la identificación de grupos significativos, sino también una mejor comprensión de las fortalezas y limitaciones de cada técnica.

Un elemento crucial fue corroborar los resultados a través de métricas internas de calidad de clúster, como Calinski-Harabasz y Davies-Bouldin. Estas herramientas resultaron fundamentales para establecer qué configuraciones reflejaban de manera más efectiva la estructura encubierta del conjunto de datos, y subrayaron que la evaluación objetiva es tan crucial como la implementación de los algoritmos.

Aparte del clustering en sí mismo, se investigó cómo el saber adquirido puede ser reutilizado en otros contextos, como en modelos supervisados. Esta relación entre técnicas no supervisadas y tareas predictivas subraya la importancia de identificar estructuras latentes como paso inicial hacia una clasificación más sólida y eficaz.

En definitiva, este estudio fortalece la capacidad de la minería de datos como un proceso completo: desde la exploración sin etiquetas, a través de la validación estricta, hasta la creación de conocimiento valioso para las fases subsiguientes. La metodología empleada puede ajustarse a diversos dominios y datasets, evidenciando que, con los instrumentos correctos, se puede transformar datos complejos en información útil para la toma de decisiones.

Referencias en formato APA:

- Caliński, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, *3*(1), 1–27. https://doi.org/10.1080/03610927408827101

- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226–231.

- Fowlkes, E. B., & Mallows, C. L. (1983). A method for comparing two hierarchical clusterings. *Journal of the American Statistical Association*, *78*(383), 553–569. https://doi.org/10.1080/01621459.1983.10478008

- Hartigan, J. A., & Wong, M. A. (1979). A K-means clustering algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, *28*(1), 100–108. https://doi.org/10.2307/2346830

- IBM. (s. f.). *¿Qué es el aprendizaje no supervisado?* IBM. Recuperado el 17 de abril de 2025, de https://www.ibm.com/mx-es/topics/unsupervised-learning

- IBM. (s. f.). *¿Qué es la minería de datos?* IBM. Recuperado el 17 de abril de 2025, de https://www.ibm.com/mx-es/topics/data-mining

- Ikotun, A. M., & Ezugwu, A. E. (2025). Cluster validity indices for automatic clustering: A comprehensive review. *PubMed*. https://pubmed.ncbi.nlm.nih.gov/39897868/

- K-means clustering algorithm. (s. f.). *NVIDIA Data Science Glossary*. Recuperado el 17 de abril de 2025, de https://www.nvidia.com/en-au/glossary/k-means/

- Naeem, S., Ali, A., Anam, S., & Ahmed, M. M. (2023). An Unsupervised Machine Learning Algorithms: Comprehensive Review. *International Journal of Computing and Digital Systems*, *13*(1). https://journal.uob.edu.bh/handle/123456789/4777

- Oti, E. U., & Olusola, M. O. (2023). Overview of Agglomerative Hierarchical Clustering Methods. *ResearchGate*. https://www.researchgate.net/publication/381265044_Overview_of_Agglomerative_Hierarchical_Clustering_Methods

- Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association*, *66*(336), 846–850. https://doi.org/10.1080/01621459.1971.10482356

- Sampaio, C. (2023, April 26). *DBSCAN with Scikit-Learn in Python*. Stack Abuse. https://stackabuse.com/dbscan-with-scikit-learn-in-python/

- Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (1998). Density-Based Clustering in Spatial Databases: The Algorithm GDBSCAN and Its Applications. *Data Mining and Knowledge Discovery*, *2*(2), 169–194.

- Steinley, D. (2006). K-means clustering: A half-century synthesis. *British Journal of Mathematical and Statistical Psychology*, *59*(1), 1–34. https://doi.org/10.1348/000711005X48266

- Yerbury, L. W., Campello, R. J. G. B., Livingston Jr, G. C., Goldsworthy, M., & O'Neil, L. (2024). On the Use of Relative Validity Indices for Comparing Clustering Approaches. *arXiv*. https://arxiv.org/abs/2404.10351
- **Basogain Olabe, X.** (s.f.). *Redes neuronales artificiales y sus aplicaciones*. Escuela Superior de Ingeniería de Bilbao, Universidad del País Vasco (UPV/EHU).
- Datacamp. (s.f.). *Tutorial de clasificación Naive Bayes con Scikit-learn*. DataCamp. https://www.datacamp.com/es/tutorial/naive-bayes-scikit-learn